PRJ301: Java Web Application Development

Slot 11, 12: Session Management

VinhVT9@fe.edu.vn

Spring 2026

FPT University Can Tho

Learning Objectives

By the end of this slot, students will be able to:

- Explain why **HTTP is stateless** and why session management is needed.
- Describe four **session tracking techniques**: URL rewriting, hidden fields, cookies, and `HttpSession`.
- Create and read **Cookies** in a Servlet; configure `maxAge`, `path`, and `domain`.
- Use `HttpSession` to store and retrieve user-specific data across requests.
- Manage the **session lifecycle**: creation, timeout, and invalidation.
- Apply session management to real-world use cases: **login** and **shopping cart**.
- Identify key **security concerns**: session hijacking, `HttpOnly`, and `Secure` flags.

FPT Education
FPT UNIVERSITY

Outline

1. HTTP Statelessness
2. Session Tracking Techniques
3. Cookies
4. HttpSession
5. Session Lifecycle
6. Cookie vs. HttpSession
7. Practical Use Cases
8. Security Considerations
9. Classwork

FPT Education
FPT UNIVERSITY

1. HTTP Statelessness

HTTP is Stateless

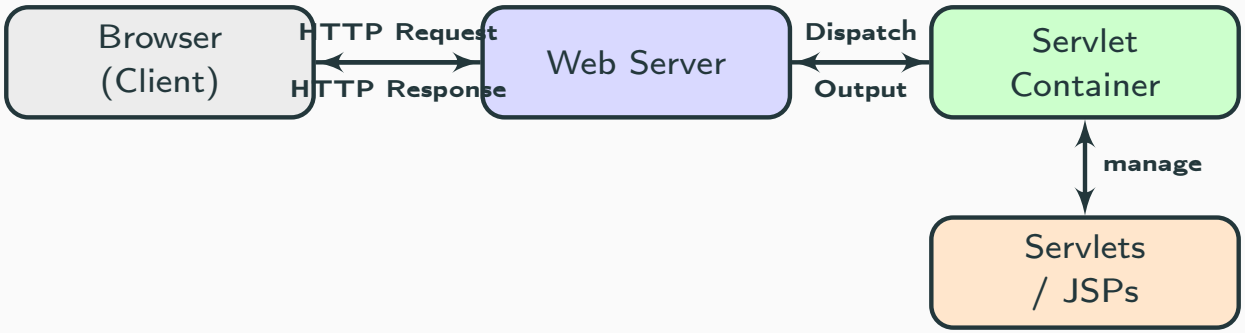
Definition
HTTP is a **stateless protocol**: each request is treated as an independent transaction with no memory of previous requests.

- The server cannot tell if request #2 comes from the same browser as request #1.
- This is by design — it makes the protocol simple and scalable.
- But most web applications **need state**: login, shopping carts, preferences, ...

The Problem
How do we recognize the **same user** across multiple HTTP requests?

FPT Education
FPT UNIVERSITY

The Web Container Model



- The **Servlet Container** (Tomcat) loads, initializes, and executes servlets.
- It handles **multiple concurrent requests** using threads.
- Each object in the container must be **thread-safe**.
- The container provides **session management** infrastructure.

FPT Education
FPT UNIVERSITY

ServletContext — Shared Application State

What is ServletContext?
A **single** `ServletContext` object exists per web application. It is shared by *all* servlets, JSPs, and listeners within the application.

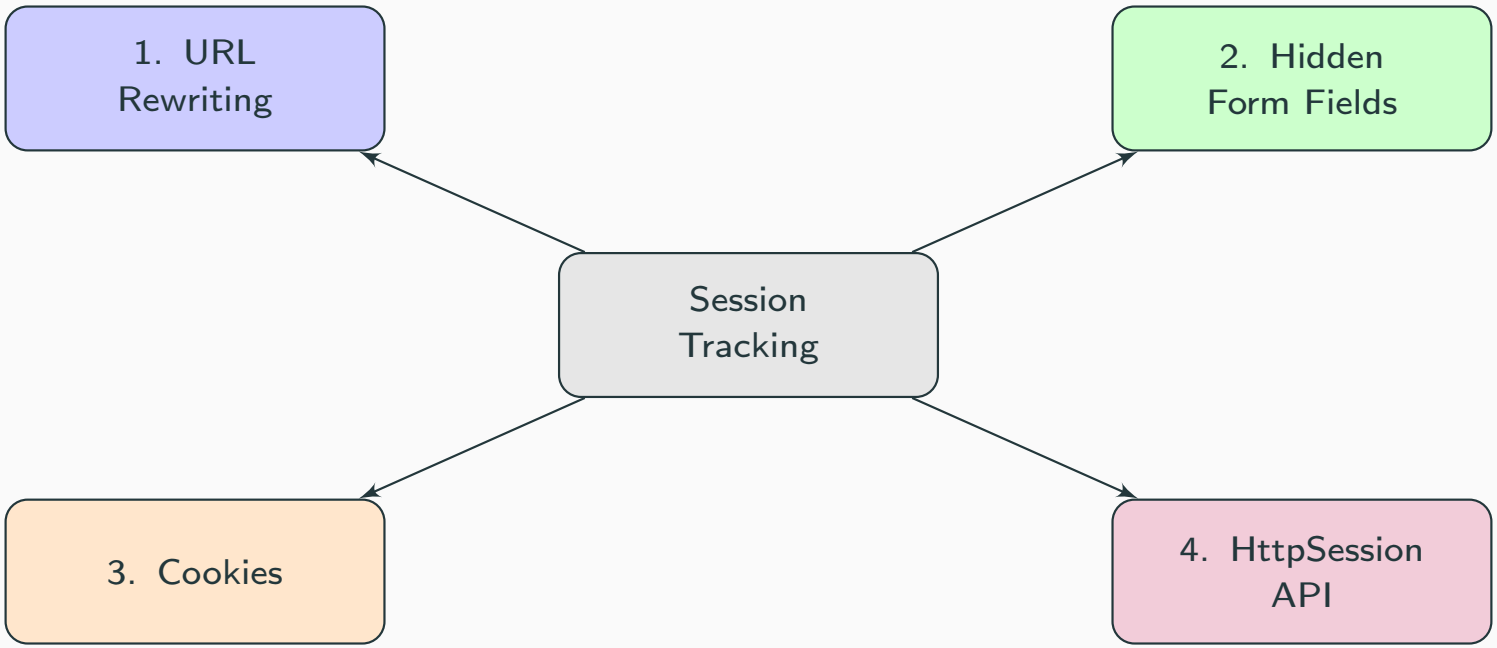
- Use it to share data **across all users and requests**.
- Access it from any servlet: `getServletContext()`.
- Initialization parameters can be set in `web.xml`:

Method	Description
`getInitParameter(name)`	Retrieve a context-wide init parameter
`setAttribute(name, obj)`	Store a shared object
`getAttribute(name)`	Retrieve a shared object
`getServletContextName()`	Application display name

FPT Education
FPT UNIVERSITY

2. Session Tracking Techniques

Four Session Tracking Techniques



The **Servlet API** officially supports **cookies** and **HttpSession**. URL rewriting and hidden fields are manual fallback strategies.

FPT Education
FPT UNIVERSITY

Technique 1: URL Rewriting

The session ID is appended as a **parameter in the URL**.

Example URL

```
http://localhost:8080/app/order;jsessionid=
58112645388D9380808A726A27F92997?item=book
```

Advantages

- Works when cookies are disabled.
- No browser support required.

Disadvantages

- Session ID is visible in browser history and server logs.
- Must be added to **every** link and form action.
- Breaks bookmarkable URLs.

FPT Education
FPT UNIVERSITY

Technique 2: Hidden Form Fields

A hidden field carries the session token in HTML forms.

```
<form action="checkout" method="POST">
  <input type="hidden" name="sessionToken"
         value="abc123xyz" />
  <!-- other form fields -->
  <input type="submit" value="Buy Now" />
</form>
```

- Works without cookies.
- Only works within **form submissions** (not plain links).
- Token must be regenerated and re-embedded on every page.
- Modern frameworks (CSRF tokens) use this pattern for **security**, not session tracking.

FPT Education
FPT UNIVERSITY

3. Cookies

What are Cookies?

Definition
A **cookie** is a small piece of data sent by the server in an HTTP response and stored by the browser. It is sent back automatically with every subsequent request to the same domain.

- Used by the Java EE container to track sessions via **JSESSIONID**.
- Can also store user preferences, language settings, etc.
- Storage: in memory (session cookie) or on disk (persistent cookie).

Attribute	Meaning
`Name / Value`	Key-value pair
`Domain`	Which domain receives this cookie
`Path`	URL path scope
`Max-Age / Expires`	How long the cookie persists
`Secure`	Only sent over HTTPS
`HttpOnly`	Not accessible from JavaScript

FPT Education
FPT UNIVERSITY

Creating and Sending Cookies

```java
@WebServlet("/setcookie")
public class SetCookieServlet extends HttpServlet {
  @Override
  protected void doGet(
      HttpServletRequest req,
      HttpServletResponse resp)
      throws ServletException, IOException {

    // Create cookie
    Cookie userCookie = new Cookie("username", "Alice");

    // Persist for 1 hour (3600 seconds)
    userCookie.setMaxAge(3600);

    // Scope: entire application
    userCookie.setPath("/");

    // Add to response
    resp.addCookie(userCookie);

    resp.getWriter().println("Cookie set!");
  }
}
```

Listing 1: Setting a cookie in a Servlet

FPT Education
FPT UNIVERSITY

Reading Cookies from a Request

```java
@WebServlet("/readcookie")
public class ReadCookieServlet extends HttpServlet {
  @Override
  protected void doGet(
      HttpServletRequest req,
      HttpServletResponse resp)
      throws ServletException, IOException {

    String username = "Guest";
    Cookie[] cookies = req.getCookies();
    if (cookies != null) {
      for (Cookie c : cookies) {
        if ("username".equals(c.getName())) {
          username = c.getValue();
          break;
        }
      }
    }
    resp.getWriter()
        .println("Hello, " + username + "!");
  }
}
```

Listing 2: Reading cookies in a Servlet

FPT Education
FPT UNIVERSITY

Cookie Attributes in Detail

Method	Effect	Example
`setMaxAge(s)`	Lifetime in seconds; `-1` = session; `0` = delete	`setMaxAge(86400)`
`setPath(p)`	Restrict to a URL path	`setPath("/app")`
`setDomain(d)`	Share across sub-domains	`setDomain(".fe.edu.vn")`
`setSecure(true)`	HTTPS only	hardening
`setHttpOnly()`	Block JS access	XSS protection

Cookie Limitations

- Maximum size: **4 KB** per cookie.
- Maximum **20–50 cookies** per domain (browser-dependent).
- Users can **disable** or delete cookies.

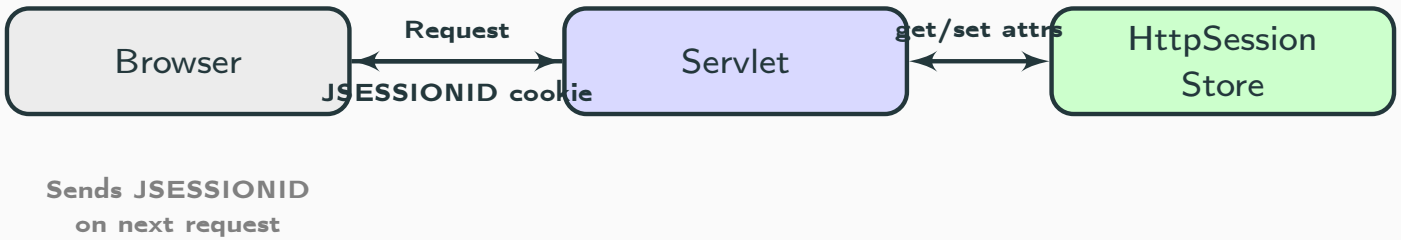
FPT Education
FPT UNIVERSITY

4. HttpSession

HttpSession: Server-Side Session

How It Works

The container creates a **session object** on the server and assigns it a unique **session ID (JSESSIONID)**. This ID is sent to the browser as a cookie (or via URL rewriting if cookies are disabled).



Key difference from cookies: **data lives on the server**, not on the client.

FPT Education
FPT UNIVERSITY

Creating and Accessing HttpSession

```java
@WebServlet("/login")
public class LoginServlet extends HttpServlet {
  @Override
  protected void doPost(
      HttpServletRequest req,
      HttpServletResponse resp)
      throws ServletException, IOException {

    String user = req.getParameter("username");
    String pass = req.getParameter("password");

    if ("admin".equals(user)
        && "1234".equals(pass)) {
      // Get existing session or create new one
      HttpSession session =
          req.getSession(true);

      // Store attribute in session
      session.setAttribute("loggedUser", user);
      session.setAttribute("role", "ADMIN");

      resp.sendRedirect("dashboard.jsp");
    } else {
      resp.sendRedirect("login.html?error=1");
    }
  }
}
```

Listing 3: Session access in a Servlet

FPT Education
FPT UNIVERSITY

Reading Session Attributes

```jsp
<%@ page contentType="text/html;charset=UTF-8"%>
<%
  HttpSession s = request.getSession(false);
  if (s == null ||
      s.getAttribute("loggedUser") == null) {
    response.sendRedirect("login.html");
    return;
  }
  String user =
    (String) s.getAttribute("loggedUser");
  String role =
    (String) s.getAttribute("role");
%>
<html><body>
  <h1>Welcome, <%= user %> [<%= role %>]</h1>
  <p>Session ID: <%= s.getId() %></p>
  <p>Created: <%= new java.util.Date(
      s.getCreationTime()) %></p>
</body></html>
```

Listing 4: dashboard.jsp - reading session data

FPT Education
FPT UNIVERSITY

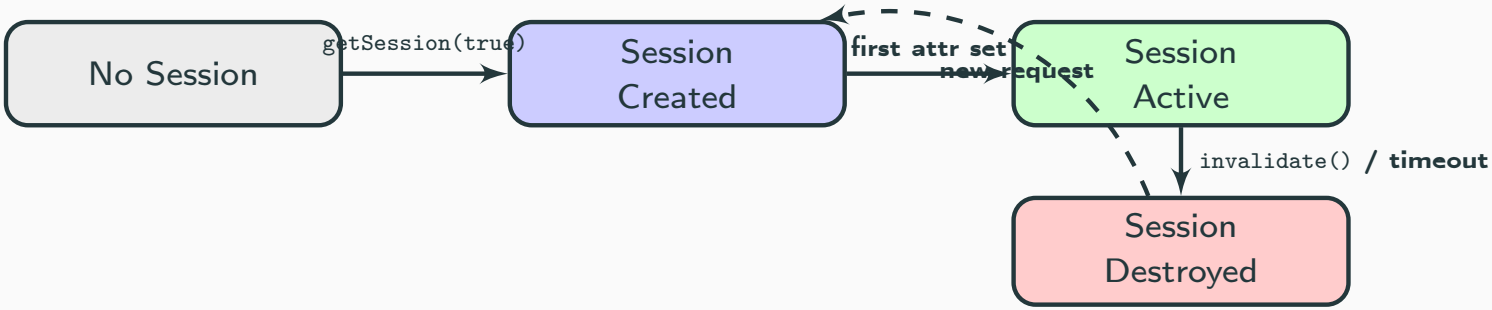
Key HttpSession Methods

Method	Description
`req.getSession()`	Get or create the session
`req.getSession(false)`	Get session; `null` if none exists
`req.getSession(true)`	Get or create (same as no-arg)
`session.getId()`	Return the session ID string
`session.setAttribute(k,v)`	Store an object
`session.getAttribute(k)`	Retrieve an object
`session.removeAttribute(k)`	Delete a stored object
`session.getAttributeNames()`	List all attribute names
`session.invalidate()`	Destroy the session
`session.getCreationTime()`	When session was created (ms)
`session.getLastAccessedTime()`	Last request time (ms)

FPT Education
FPT UNIVERSITY

5. Session Lifecycle

Session Lifecycle



Three ways a session ends:

1. **Explicit invalidation:** `session.invalidate()` (e.g., logout).
2. **Timeout:** No request received within the inactivity timeout.
3. **Container shutdown:** Server stops (non-persistent sessions).

FPT Education
FPT UNIVERSITY

Session Timeout Configuration

Option 1: web.xml — application-wide default

```
<web-app>
  <session-config>
    <!-- timeout in minutes; 0 = never -->
    <session-timeout>30</session-timeout>
  </session-config>
</web-app>
```

Option 2: Per-session in Java code

```
HttpSession session = req.getSession();
// Set timeout in seconds; <=0 means never
session.setMaxInactiveInterval(1800);
```

Priority
Per-session `setMaxInactiveInterval()` **overrides** the `web.xml` default for that individual session.

FPT Education
FPT UNIVERSITY

Session Invalidation (Logout)

```java
@WebServlet("/logout")
public class LogoutServlet extends HttpServlet {
  @Override
  protected void doGet(
      HttpServletRequest req,
      HttpServletResponse resp)
      throws ServletException, IOException {

    // Retrieve session without creating new one
    HttpSession session =
        req.getSession(false);

    if (session != null) {
      // Remove individual attribute
      session.removeAttribute("loggedUser");

      // OR destroy entire session
      session.invalidate();
    }

    // Redirect to login page
    resp.sendRedirect("login.html");
  }
}
```

Listing 5: LogoutServlet.java

FPT Education
FPT UNIVERSITY

6. Cookie vs. HttpSession

Cookies vs. HttpSession: Comparison

Feature	Cookie	HttpSession
Storage location	**Client-side** (browser)	**Server-side** (memory)
Data size limit	4 KB per cookie	Unlimited (limited by server RAM)
Security	Lower (data on client)	Higher (data on server)
Performance	Less server memory	Uses server memory per user
Lifetime	Configurable (long)	Usually short (timeout)
Requires cookie support	Yes (to transmit ID)	Yes (JSESSIONID) or URL rewriting
Best used for	Preferences, language, tokens	Login state, cart, sensitive data

Rule of Thumb

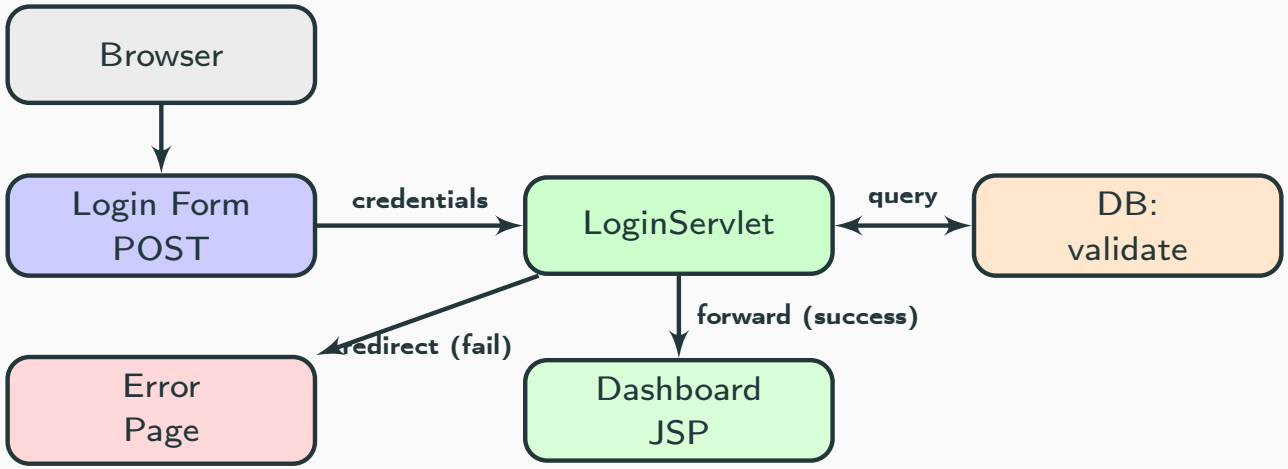
Store **sensitive state** (login, roles) in `HttpSession`.

Store **preferences** (theme, language) in cookies.

FPT Education
FPT UNIVERSITY

7. Practical Use Cases

Use Case 1: Login Session



1. User submits credentials via form.
2. Servlet validates against database.
3. On success: `session.setAttribute("user", userObj)`.
4. Subsequent pages check `session.getAttribute("user") != null`.
5. Logout: `session.invalidate()`.

FPT Education
FPT UNIVERSITY

Use Case 2: Shopping Cart

```java
@WebServlet("/cart")
public class CartServlet extends HttpServlet {
  @Override
  protected void doPost(
      HttpServletRequest req,
      HttpServletResponse resp)
      throws ServletException, IOException {

    String item = req.getParameter("item");
    HttpSession session = req.getSession();

    // Retrieve or create cart list
    @SuppressWarnings("unchecked")
    List<String> cart = (List<String>)
        session.getAttribute("cart");
    if (cart == null) {
      cart = new java.util.ArrayList<>();
      session.setAttribute("cart", cart);
    }

    cart.add(item);
    resp.sendRedirect("cart.jsp");
  }
}
```

Listing 6: CartServlet.java (simplified)

FPT Education
FPT UNIVERSITY

8. Security Considerations

Security: Session Hijacking

Session Hijacking

An attacker steals a user's **session ID** and uses it to impersonate them.

Common attack vectors:

- **XSS (Cross-Site Scripting):** Malicious JavaScript reads `document.cookie`.
- **Network sniffing:** Intercept JSESSIONID in plaintext HTTP traffic.
- **URL exposure:** Session ID visible in browser history or server logs.
- **Session fixation:** Attacker provides a known session ID before login.

Defense

Regenerate the session ID after login:

`req.getSession(false).invalidate()` then `req.getSession(true)`.

FPT Education
FPT UNIVERSITY

Security: HttpOnly and Secure Flags

HttpOnly: Prevents JavaScript from reading the cookie.

- Protects against **XSS attacks** that steal session cookies.

Secure: Cookie is only sent over **HTTPS**.

- Prevents **network sniffing** on plain HTTP connections.

Configure in web.xml (Servlet 3.0+):

```
<web-app>
  <session-config>
    <cookie-config>
      <http-only>true</http-only>
      <secure>true</secure>
    </cookie-config>
    <session-timeout>30</session-timeout>
  </session-config>
</web-app>
```

Best Practice
Always enable **HttpOnly** and **Secure** in production.

Security: Setting Flags Programmatically

For individual cookies in Java code:

```
Cookie c = new Cookie("PREF", "dark-mode");
c.setHttpOnly(true);   // No JS access
c.setSecure(true);     // HTTPS only
c.setMaxAge(604800);   // 7 days
c.setPath("/");
resp.addCookie(c);
```

Additional security best practices:

- **Never** store passwords or sensitive data in cookies.
- Use **short session timeouts** for sensitive applications (banking: 5–15 min).
- **Validate session** on every protected page (`getSession(false) != null`).
- Regenerate session ID on **privilege escalation** (after login).
- Use **HTTPS** for all pages that involve session cookies.

FPT Education
FPT UNIVERSITY

9. Classwork

Classwork 5: Session Management Tasks

Task A: Cookie-Based Preference

1. Create a form with a dropdown: **Theme** (light / dark).
2. On submit, a Servlet saves the preference as a **persistent cookie** (7 days).
3. On every page load, read the cookie and apply the CSS class accordingly.

Task B: Login + Protected Page

1. Build a login form that POSTs to `LoginServlet`.
2. On success, store the username in **HttpSession** and redirect to `dashboard.jsp`.
3. `dashboard.jsp` must check the session; redirect to `login.html` if no session.
4. Add a **Logout** link that calls `LogoutServlet` to invalidate the session.

Task C: Shopping Cart

1. Create a product list page with "Add to Cart" buttons.
2. Use `HttpSession` to maintain an `ArrayList` of cart items.
3. Display the cart contents and allow removing items.

FPT Education
FPT UNIVERSITY

Classwork 5: Discussion Questions

1. Why is **HTTP stateless**? What would be different if HTTP were stateful?
2. What is the difference between `req.getSession()` and `req.getSession(false)`? When should you use each?
3. A user disables cookies in their browser. How does Tomcat handle session tracking in this case? What are the trade-offs?
4. Explain the difference between **session timeout** (web.xml) and `setMaxInactiveInterval()`. Which takes priority?
5. Why is it important to call `session.invalidate()` on logout rather than just removing the user attribute?
6. What is **session fixation**? Describe one defense against it in a servlet.

FPT Education
FPT UNIVERSITY

Summary: Slot 11, 12 — Session Management

- **HTTP is stateless** — session management bridges this gap.
- **4 techniques**: URL rewriting, hidden fields, cookies, HttpSession (servlet API supports last two officially).
- **Cookies**: client-side, 4 KB limit, key attributes: `maxAge`, `path`, `domain`, `HttpOnly`, `Secure`.
- **HttpSession**: server-side, identified by JSESSIONID cookie; use `setAttribute`/`getAttribute` to store data.
- **Session lifecycle**: created on `getSession(true)`, destroyed via `invalidate()` or timeout.
- **Timeout**: configured in `web.xml` (minutes) or `setMaxInactiveInterval()` (seconds).
- **Security**: enable `HttpOnly` + `Secure` flags; use HTTPS; regenerate session ID after login.
- **Use cases**: login sessions, shopping carts, user preferences.

Up Next
Slot 13, 14: Expression Language (EL) and JSTL — cleaner JSP without scriptlets.

FPT Education
FPT UNIVERSITY

Questions?

FPT Education
FPT UNIVERSITY